Exp-7:-

15/9/26

Sliding Window Protocol

Aim:- write a program to implement flow control at data link layer using sliding window protocol. Simulate the flow of ~~famous~~ frames. frome one node to another

Description:-

- Flow control prevents fast sender from overwhelming a slow receiver by restricting the number of un-acknowledged frames to a specific window size.
- Sender transmits upto W frames and saves to Sender-Buffer.
- Receiver checks sequence number validity. if valid, sends ~~Ack~~ else sends NACK with reverse data (piggy backing) via receiver-buffer.
- On ACK, sender slides window forward on NACK to retransmit from failed frame.

Algorithm:-

Sender:-

1) Input window size W and text message
2) Transmit window frames [frame-no. data] into Sender_Buffer.txt
3) Wait and read Receiver_Buffer.txt for response.

4) If ACK, slide window forward (base = ack_no); if NACK, reset base to retransmit.

Receiver :-

1) Poll sender_Buffer.txt and parse frame sequence numbers.

2) if frame matches expect, increment counter and return ACK with piggybacked character.

3) If frame mismatch/corrupted, write NACK with expected frame number to Receiver_Buffer.txt.

Program code :-

(i) sender.py :-

```
import os, time
w = int(input("Enter window size: "))
msg = input("Enter text message: ")
base, n = 0, len(msg)
while base < n:
    end = min(base + w, n)
    frames = [f"{i},{msg[i]}" for i in range(base, end)]
with open("Sender_Buffer.txt", "w") as f:
    f.write("\n".join(frames))
print(f"\n[TX] Sent frames {base} to {end-1}")
while not os.path.exists("Receiver_Buffer.txt") or
    os.stat("Receiver_Buffer.txt").st_size == 0:
    time.sleep(1)
```

```
with open("Receiver_Buffer.txt", "r") as f:
    reply = f.read().strip().split(",")
os.remove("Receiver_Buffer.txt")

kind, num = reply[0], int(reply[1])
if len(reply) > 2:
    print(f"Piggybacked data received: {reply[2]}")
if kind == "ACK":
    print(f"ACK {num} received → Window slides")
    base = num
else:
    print(f"NACK {num} received → Resending from
    frame {num}")
    base = num

print("\n All frames delivered successfully.")
```

ii) Receiver.py

```
import os, time
reply_msg = input("Enter IP")
expected, ~~pos, rec_data~~ = 0, 0,
print("waiting")
while True:
    if not os.path.exists("Sender_Buffer.txt"):
        time.sleep(1)
        continue
    with open("Sender_Buffer.txt", "r") as f:
        lines = f.read().splitlines()
    os.remove("Sender_Buffer.txt")
```

```
err=False
for line in lines:
    num, ch= line.split(",")
    if int(num) == expected:
        print(f"Frame {num} [{ch}] accepted")
        expected +=1
    else:
        print(f" Wrong frame {num}, expected
                                {expected}")
        err = True
        break
piggy = reply[p] if p<len(reply) else ""
if not err:
    resp = f"ACK,{expected},{piggy}"
    p+=1
else:
    resp = f"NACK,{expected},{piggy}"

with open("Receiver_Buffer.txt","w") as f:
    f.write(resp)
print("Sent:",resp)
```

Output:-

Sender:-

3
ABC

[TX] sent frames 0 to 2
Frame No : 0 Data : A
Frame No : 1 Data : B
Frame No : 2 Data : C

Receiver :-

Frame 0 [A] accepted
Wrong Frame 5, expected 1
Sent: NACK, 1, X

Sender Resend :-

NACK received : 1
Resending from Frame 1

Result :-

The sliding window protocol with error detection, retransmission and piggybacking was verified successfully.

13/7/22